VENDOR PERFORMANCE ANALYSIS

An End-to-End Data Analytics Pipeline — SQL ETL, Feature Engineering, EDA & Statistical Inference

Python

SQLite

Pandas / Seaborn

ETL Pipeline

SciPy Stats

Prepared by: Data Analyst | 126 Vendors • 10,692 Vendor–Brand Records • $451.6M in Tracked Sales

Business Problem & Dataset Overview

The Business Problem

- A multi-vendor liquor distribution business tracks purchases, sales, freight and invoices across separate systems.
- Without a unified view, management cannot see which vendors are profitable, which products sit as dead stock, or where pricing is misaligned with cost.
- This project consolidates 4 raw data sources into one analytical model to answer: Which vendors and brands drive profit — and which drain it?

Dataset Overview

4
Raw SQLite Tables
purchases · purchase_prices · sales · vendor_invoice

10,692
Vendor–Brand Records
consolidated via CTE-based SQL joins

126
Unique Vendors
spanning national & regional suppliers

10,663
Unique Brand Items
individual product-level detail

Why It Matters: Every 1% improvement in profit margin across $451.6M in tracked sales represents roughly $4.5M in recovered margin.

End-to-End Pipeline Architecture

CSV Source Files	Raw purchase, sales & invoice extracts
SQLite Database	Ingested via chunked pandas.to_sql()
SQL (CTEs & Joins)	Purchases + Prices + Sales + Freight merged
Python ETL Pipeline	Cleaning, type-casting, deduplication
Logging Layer	Every stage timestamped & audit-logged
Feature Engineering	GrossProfit, Margin, StockTurnover, Ratio
Exploratory Data Analysis	Distributions, correlation, Pareto, rankings
Statistical Analysis	95% confidence intervals, hypothesis testing
Business Insights	Vendor risk, pricing & inventory decisions

Data Engineering: Ingestion & Logging

```
ingestiondb.py



engine = create_engine("sqlite:///inventory.db")

def ingest_db(df, table_name, engine, mode):
    df.to_sql(name=table_name, con=engine,
              if_exists=mode, index=False)

def load_raw_data():
    for file in os.listdir():
        if file.endswith(".csv"):
            table_name = os.path.splitext(file)[0]
            for chunk in pd.read_csv(file,
                    chunksize=10000):
                ingest_db(chunk, table_name, engine,
                    "replace" if first_chunk else "append")
                total_rows += len(chunk)
            logging.info(f"{table_name} imported | "
                         f"Rows: {total_rows:,}")
```

How the Pipeline Runs

- **Chunked Ingestion**

  CSVs load in 10,000-row chunks — memory-safe for large files, first chunk replaces the table, later chunks append.

- **Modular Design**

  ingest_db() is a single reusable function, imported into downstream scripts (get_vendor_summarry.py) rather than duplicated.

- **Full Audit Logging**

  logging module timestamps every stage to ingestion_db.log — start, per-file row counts, and total runtime in minutes.

- **Reproducible ETL**

  SQLAlchemy engine + if_exists mode make re-runs idempotent — safe to re-ingest without duplicating data.

Data Cleaning & Feature Engineering

Raw Data Issues

- Volume stored as text/object dtype
- Null values in freight & sales fields
- Leading/trailing spaces in VendorName & Description
- No standard profitability metric across tables

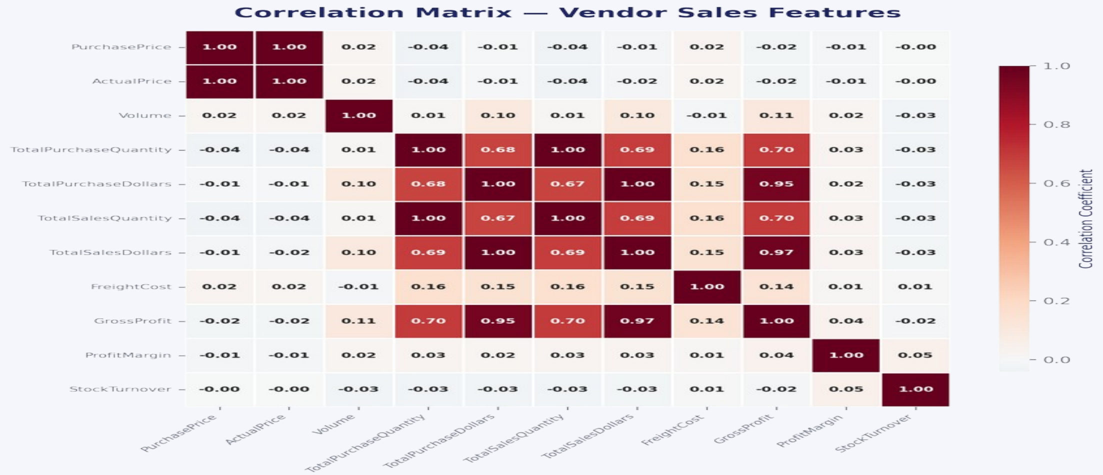
Cleaning Applied

- Volume cast to float64 for numeric ops
- fillna(0) applied across the summary table
- VendorName / Description stripped of whitespace
- Consolidated into one vendor_sales_summary table

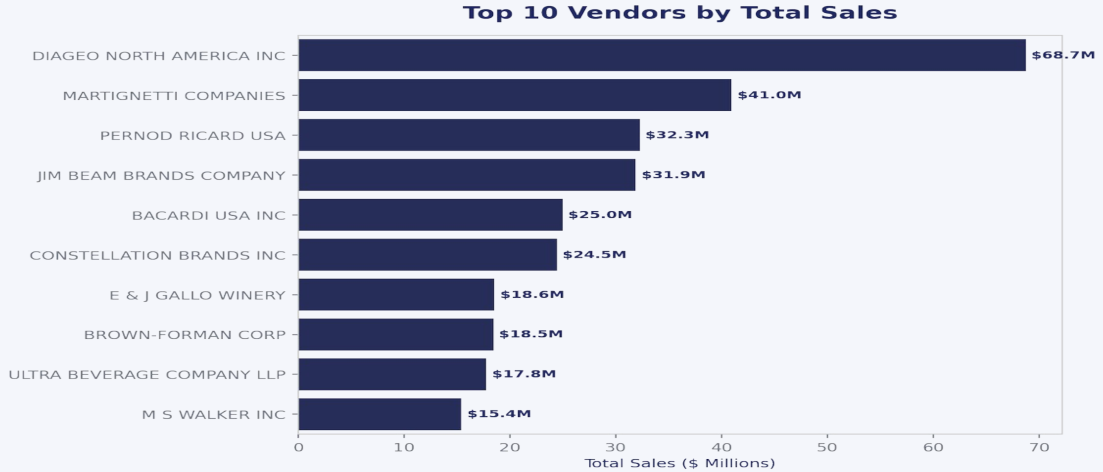
Engineered Metrics

- GrossProfit = Sales − Purchases
- ProfitMargin = GrossProfit / Sales × 100
- StockTurnover = Sales Qty / Purchase Qty
- SalesToPurchaseRatio = Sales$ / Purchase$

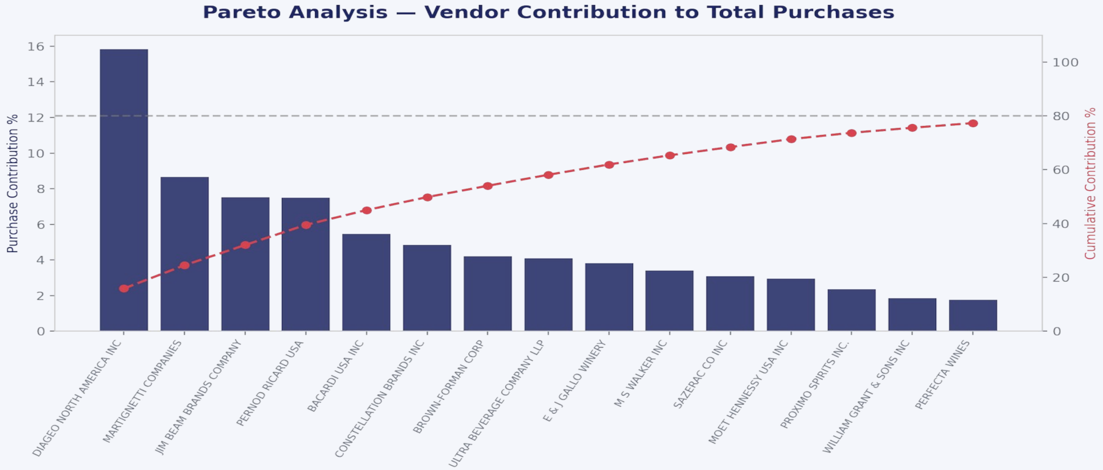
Exploratory Data Analysis



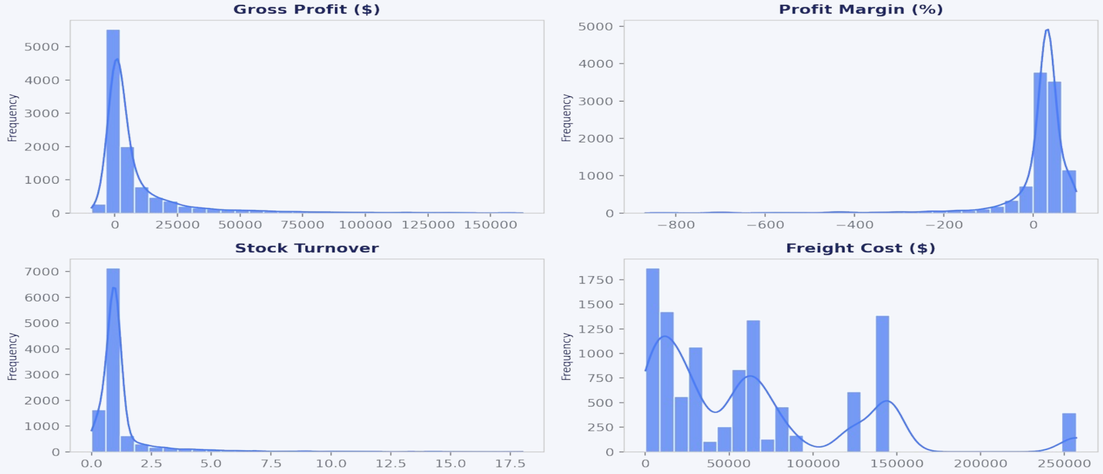
Correlations: Purchase $ and Sales $ move together; margin is largely independent of volume.



Diageo North America leads all vendors at $68.7M in tracked sales.

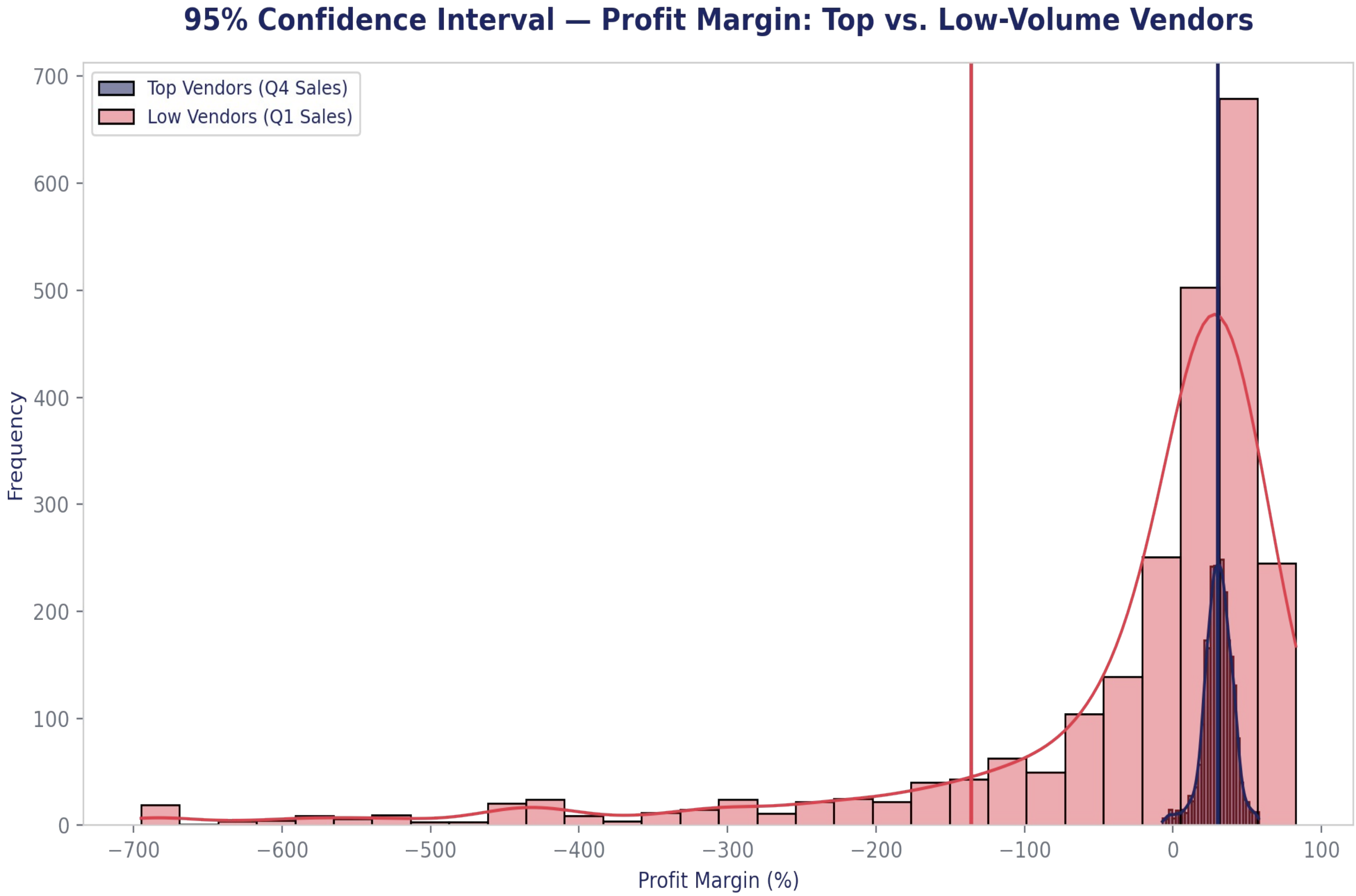


~15 vendors already account for the bulk of purchase spend (80/20 pattern).



Profit margin and stock turnover are both heavily right-skewed with a long loss-making tail.

Statistical Analysis: Confidence Intervals



Top-Quartile Vendors

30.0%

95% CI: 29.5% – 30.5%   (n = 2,629)

Tight, stable margins — high-volume vendors price consistently and predictably.

Bottom-Quartile Vendors

-136.0%

95% CI: -169.4% – -102.5%   (n = 2,630)

Wide, deeply negative interval — low-volume vendors show erratic, often loss-making pricing.

Method: Vendors split into top vs. bottom sales quartiles; mean profit margin and its 95% confidence interval computed for each group using Student's t-distribution (scipy.stats). The non-overlapping intervals indicate the margin gap between high- and low-volume vendors is statistically meaningful, not noise.

Business Insights

$451.6M

Total Sales Tracked

$129.7M

Total Gross Profit

30.8%

Median Profit Margin

178 SKUs

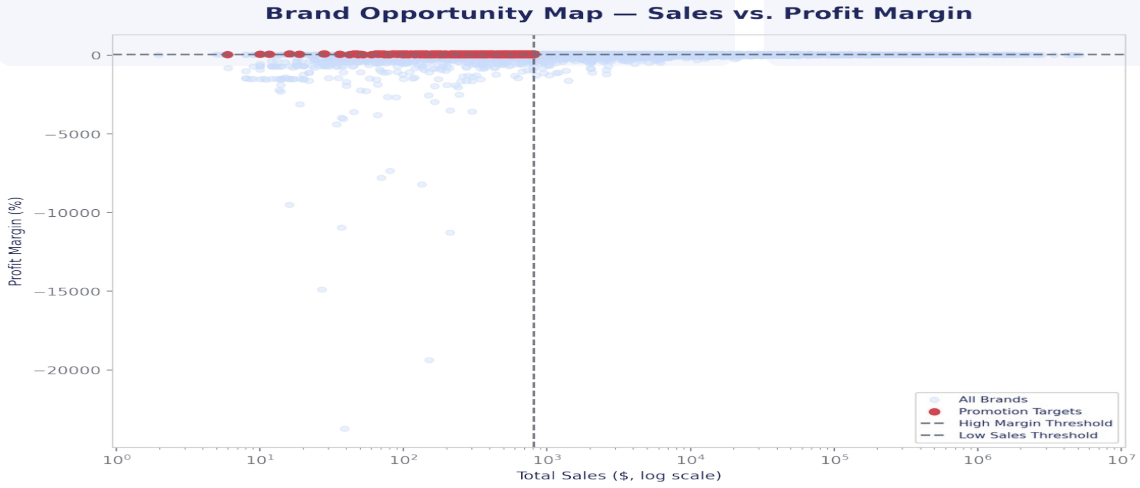
Zero-Sale (Dead) Inventory

19.9%

Line Items Sold at a Loss

65.0%

Sales Held by Top 10 Vendors



520 brands

sit in the low-sales / high-margin quadrant — strong candidates for promotional investment to unlock volume without sacrificing margin.

Recommendations

Clear Dead Inventory

178 SKUs (1.7% of items) have zero sales, tying up $282K in purchase value. Liquidate or discontinue to free working capital.

Fix Loss-Making Pricing

19.9% of line items (2,127 SKUs) sell below cost, a combined $4.35M drag on gross profit. Prioritize repricing the largest offenders first.

Reduce Vendor Concentration Risk

Top 10 vendors supply 65% of purchases and sales. Diversify sourcing or negotiate resilience clauses to limit single-vendor disruption risk.

Promote High-Margin, Low-Volume Brands

520 brands combine strong margin with weak sales volume — the highest-ROI segment for targeted marketing spend.

Negotiate with Pareto-Priority Vendors

~15 vendors drive 80% of purchase spend — consolidate negotiating leverage and freight terms with this group first.

Stand Up Ongoing Monitoring

Automate this pipeline (ingestion → SQL → EDA → CI testing) into a recurring dashboard to catch margin erosion and dead stock earlier.

Project Summary

Skills Demonstrated

- End-to-end ETL design: chunked ingestion, SQLAlchemy, modular Python
- Advanced SQL: CTEs, multi-table joins, aggregations at scale
- Data cleaning & feature engineering for business KPIs
- EDA storytelling with Pandas, Seaborn & Matplotlib
- Inferential statistics: confidence intervals with SciPy
- Production hygiene: logging, error handling, reproducibility

Business Impact & What's Next

Turned 4 disconnected raw tables into a single model surfacing $4.35M in loss-making sales, $282K in dead stock, and a 520-brand growth opportunity.

- Automate this pipeline into a scheduled, refreshing dashboard
- Extend the T-test / CI framework to formal hypothesis tests across vendor cohorts
- Add demand forecasting to flag future dead stock before it accumulates
- Deploy as an interactive Power BI / Streamlit report for stakeholders

github.com/your-username/vendor-performance-analysis • Thank you — questions welcome